

09:00-09:50 (쉬는 시간 11:30-12:00)

Meeting·Review Service Router

Router · Input kind · unified_service_result.json

진행

Meeting·Review Service Router
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/service_router.py
src/course_services/
meeting_service.py

확인할 결과

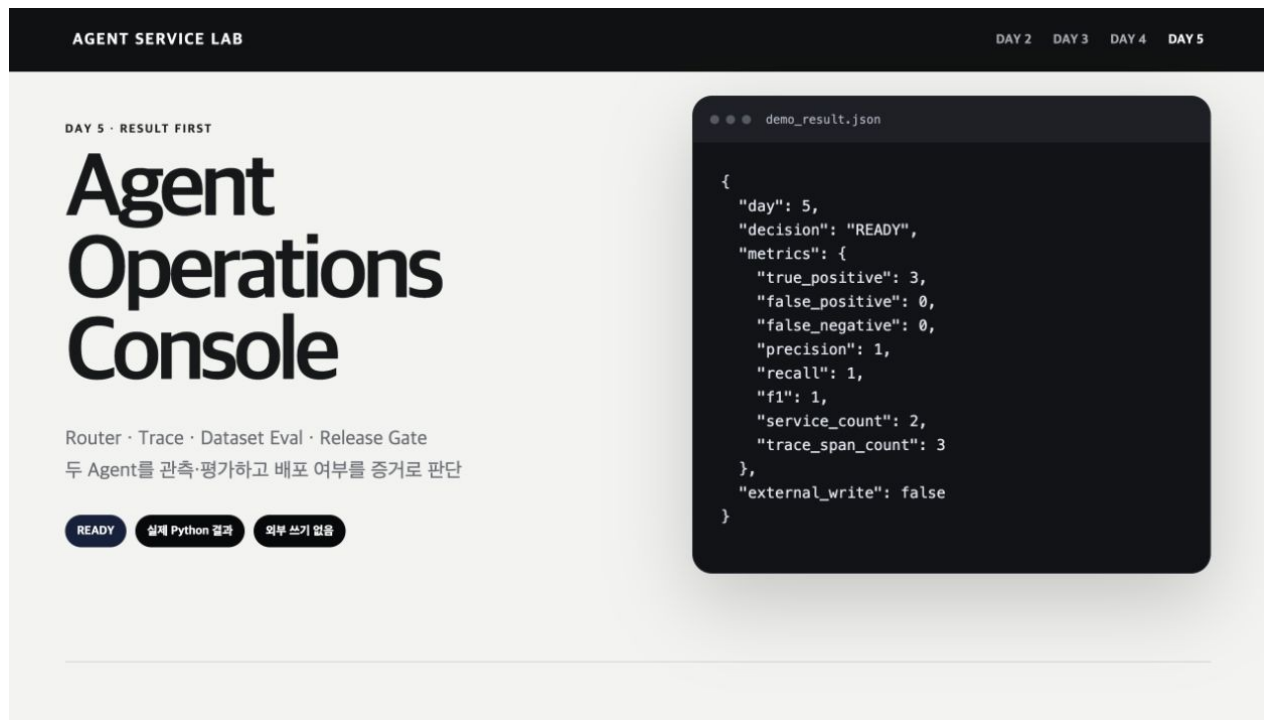
unified_service_result.json

Agent Operations Console 완성 화면

모델 추측으로 업무 종류를 고르면
잘못된 tool과 권한이 연결될 수 있다.

입력 계약

- 핵심 처리
- 실패 경계
- 사람 판단
- 평가·실행 증거



재생 파일 · assets/demo-videos/day5_service_teaser.gif

오늘 8차시는 이 화면을 현재 저장소에서 다시 만드는 과정입니다.

5일차 전체 시간표

3시간 블록(30분 휴식) · 2시간 블록(20분 휴식) · 점심 14:00-15:00 · 3시간 블록(30분 휴식)

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	Meeting·Review Service Router	강의 12 시연 10 Codex 제작 23 확인 5분	unified_service_result.json
09:50-10:40	2차시	LangSmith Trace 구조	강의 12 시연 10 Codex 제작 23 확인 5분	trace.json 또는 LangSmith run
10:40-11:30	3차시	운영 Metadata와 Monitoring	강의 12 시연 10 Codex 제작 23 확인 5분	monitoring_view.md
12:00-12:50	4차시	Dataset Experiment와 Release Gate	강의 12 시연 10 Codex 제작 23 확인 5분	experiment_result.json
12:50-13:40	5차시	Human Feedback과 Annotation	강의 12 시연 10 Codex 제작 23 확인 5분	human_feedback.jsonl
15:00-15:50	6차시	PII·Retention·Incident 운영	강의 12 시연 10 Codex 제작 23 확인 5분	ops_checklist.md
15:50-16:40	7차시	Release Candidate와 Vercel Demo	강의 12 시연 10 Codex 제작 23 확인 5분	release_candidate
16:40-17:30	8차시	최종 Demo와 Scorecard	강의 12 시연 10 Codex 제작 23 확인 5분	final_tag_and_scorecard

11:30-12:00 쉬는 시간 · 13:40-14:00 쉬는 시간 · 14:00-15:00 점심시간 · 17:30-18:00 쉬는 시간

Meeting·Review Service Router 학습 결과

- 01 — meeting_transcript와 code_diff를 같은 entry point에서 실행한다.
- 02 — 내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다. 왜 위험한지 설명한다.
- 03 — unified_service_result.json에서 정상과 실패 상태를 직접 확인한다.

Meeting-Review Service Router 용어

용어	이 수업에서 쓰는 뜻
Router	입력을 알맞은 workflow로 전달
Input kind	호출자가 명시한 업무 종류
Adapter	서로 다른 service 결과를 연결
Boundary	읽기·쓰기·승인 경계

unified_service_result.json 입출력

INPUT

src/course_services/service_router.py

OUTPUT

unified_service_result.json

PROCESS

path validate → input kind 확인 → meeting 또는 review

VERIFY

회의와 diff가 각각 올바른 service 이름으로 반환된다.
unknown kind와 workspace escape는
EXPECTED_FAILURE다.

Meeting-Review Service Router 실행 흐름

01

path validate

02

input kind 확인

03

meeting 또는
review

04

공통 result

05

사람 승인

마지막 판단은 unified_service_result.json에서 사람이 확인합니다.

unified_service_result.json 정상·실패 계약

정상	——	회의와 diff가 각각 올바른 service 이름으로 반환된다.
경계	——	unknown kind와 workspace escape는 EXPECTED_FAILURE다.
외부 쓰기	——	Meeting·Review Service Router: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·unified_service_result.json

Meeting-Review Service Router 파일 지도

입력·fixture

`src/course_services/service_router.py`

구현 코드

`src/course_services/meeting_service.py`

검증·test

`src/course_services/review_service.py`

따라하기 notebook

`materials/day5/day5_service_lab.ipynb`

Meeting·Review Service Router 개념·코드

Router

입력을 알맞은 workflow로 전달

```
src/course_services/service_router.py
```

Input kind

호출자가 명시한 업무 종류

```
src/course_services/meeting_service.py
```

Adapter

서로 다른 service 결과를 연결

```
src/course_services/review_service.py
```

Boundary

읽기·쓰기·승인 경계

```
materials/day5/day5_service_lab.ipynb
```

Meeting-Review Service Router 선택 기준

구분	역할	이번 차시의 판단 기준
Router	path validate 단계의 입력 기준	결정론 test로 먼저 확인
Input kind	input kind 확인 단계의 교체 경계	adapter 경계를 확인
Adapter	meeting 또는 review 단계의 운영 조건	비용·지연·권한을 확인
Boundary	공통 result 단계의 판단 기록	사람 판단과 운영 기록을 확인

Meeting-Review Service Router 대표 실패

실패

내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다.

사용자 영향

내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다. 상태에서는 unified_service_result.json을 운영 판단에 사용할 수 없다.

Meeting-Review Service Router 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Meeting·Review Service Router 복구 경로

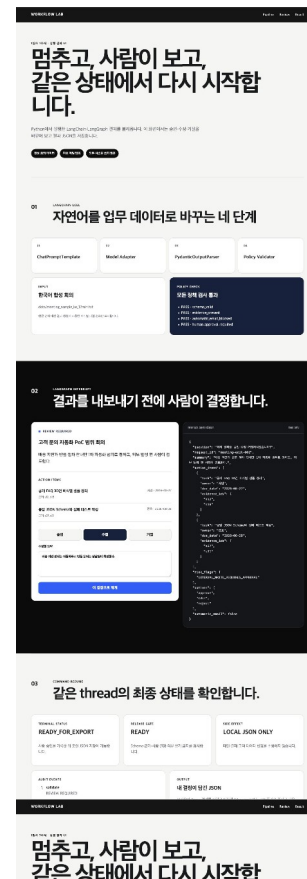
- 01 — 탐지: unknown kind와 workspace escape는 EXPECTED_FAILURE다.
- 02 — 중단: 내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다.
- 03 — 복구: input_kind를 API contract로 받고 unsupported type은 거부한다.
- 04 — 증거: unified_service_result.json과 test 결과를 함께 남긴다.

Meeting-Review Service Router Codex 검증 루프

src/course_services/
meeting_service.py의 책임을
찾고
unified_service_result.json
계약을 focused test로 고정합니다.

Meeting-Review Service Router

저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Meeting·Review Service Router Codex 작업 명세

목표

새 `service route`를 추가할 때 `workspace` 경계와 `external_write=false`를 `test`한다.

허용 경로

`src/course_services/meeting_service.py`
`src/course_services/review_service.py`

완료 조건

회의와 `diff`가 각각 올바른 `service` 이름으로 반환된다.
`unknown kind`와 `workspace escape`는 `EXPECTED_FAILURE`다.

금지

Meeting·Review Service Router: `secret` 출력 · `workspace` 밖 변경 · `test` 약화 · 승인 없는 외부 쓰기

Meeting·Review Service Router 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | Meeting·Review Service Router 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 회의와 diff가 각각 올바른 service 이름으로 반환된다. |
| 03 | Claude review | —— | unified_service_result.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Meeting·Review Service Router의 두 LLM 의견과 test는 자동 승인이 아니다 |

Meeting-Review Service Router 정상 시연

- 01 — 열기: `src/course_services/service_router.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k router`
- 04 — 성공 신호: 회의와 diff가 각각 올바른 service 이름으로 반환된다.

Meeting-Review Service Router 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k router
```

실행 전 확인

Meeting-Review Service Router: repository root · Python 3.12 · .env 미출력

01_unified_service_result.json 실행 결과

```
{
  "meeting": {
    "status": "SUCCESS",
    "service": "meeting",
    "result": {
      "title": "고객 문의 자동화 PoC 범위 회의",
      "summary": "배송 지연·반품 안내를 1차 범위로 정하고 외부 발행 전 휴먼 검토를 적용한다.",
      "decisions": [
        "1차 자동화 범위는 배송 지연 문의와 반품 절차 안내로 제한한다.",
        "개인정보·낮은 정확도·근거 없음은 상담원 검토 큐로 보낸다."
      ],
      "action_items": [
        {
          "task": "공개 FAQ 30건 비식별 샘플 정리",
          "owner": "서연",
          "due_date": "2026-08-27"
        },
        {
          "task": "응답 JSON 스키마와 실패 테스트 작성",
          "owner": "준호",
          "due_date": "2026-08-28"
        }
      ],
      "evidence_line_count": 7,
      "generated_on": "2026-08-25",
      ...
    }
  }
}
```

Meeting·Review Service Router 실패·복구 시연

재현

unknown kind와 workspace escape는
EXPECTED_FAILURE다.

복구

input_kind를 API contract로 받고
unsupported type은 거부한다.

확인: unknown kind와 workspace escape는 EXPECTED_FAILURE다.

Meeting·Review Service Router 소프트웨어 제작

열 파일

`src/course_services/meeting_service.py`
`src/course_services/review_service.py`

작업 범위

Meeting·Review Service Router 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

회의와 diff가 각각 올바른 service 이름으로 반환된다.
`unified_service_result.json` 저장

Meeting-Review Service Router Notebook 셀

materials/day5/day5_service_lab.ipynb · 1차시

```
from src.course_services.service_router import route_service_request

meeting = route_service_request(
    input_kind="meeting_transcript", source_path=R00T / "data/meeting_sample_ko.txt", workspace_root=R00T,
)
review = route_service_request(
    input_kind="code_diff", source_path=R00T / "data/day3_review_cases/unsafe_pr.diff", workspace_root=R00T,
)
unknown = route_service_request(
    input_kind="unknown", source_path=R00T / "data/meeting_sample_ko.txt", workspace_root=R00T,
)
router_result = {"meeting": meeting, "review": review, "boundary": unknown}
assert unknown["error_code"] == "UNSUPPORTED_INPUT_KIND"
save_json("01_unified_service_result.json", router_result)
{"meeting": meeting["service"], "review": review["service"], "boundary": unknown["error_code"]}
```

Meeting-Review Service Router Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k router
```

저장 위치

```
output/course-labs/day5/01_unified_service_result.json
```

회의와 diff가 각각 올바른 service 이름으로 반환된다.
실패 시 unified_service_result.json의 status·error_code를 먼저 확인

Meeting·Review Service Router 실패 증거

unknown kind와 workspace escape는 EXPECTED_FAILURE다.

```
result_file = "output/course-labs/day5/01_unified_service_result.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

unified_service_result.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Meeting·Review Service Router 정상 Case

NORMAL

회의와 diff가 각각 올바른 service 이름으로 반환된다.

- 01 — src/course_services/service_router.py 입력과 command가 기록됐는가
- 02 — unified_service_result.json schema가 validate되는가
- 03 — Meeting·Review Service Router focused test가 실제로 통과했는가

Meeting·Review Service Router 경계 Case

BOUNDARY

unknown kind와 workspace escape는 EXPECTED_FAILURE다.

- 01 — Meeting·Review Service Router: raw traceback 대신 stable error_code가 남는가
- 02 — unified_service_result.json: 외부 쓰기·자동 메일이 false인가
- 03 — unknown kind와 workspace escape는 EXPECTED_FAILURE다. 재현이 가능한가

Meeting·Review Service Router 현업 적용

한 화면에서 회의·PR 리뷰를 선택 실행하는 내부 도구

- 01 — Meeting·Review Service Router은 합성·비식별 sample로 먼저 실행
- 02 — 한 화면에서 회의·PR 리뷰를 선택 실행하는 내부 도구의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — unified_service_result.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — unified_service_result.json을 운영 검토 자료로 사용

Meeting·Review Service Router 포트폴리오 적용

Meeting·Review Service Router · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다.의 사용자 영향을 한 문장으로 설명
- 02 ——— Meeting·Review Service Router 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 새 service route를 추가할 때 workspace 경계와 external_write=false를 test한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— unified_service_result.json과 README 재현 절차를 제출

Meeting·Review Service Router 운영 점검

품질

무엇을 회의와 diff가 각각 올바른 service 이름으로 반환된다.로 정의하는가

복구

내용을 LLM에게 보여주고 업무 종류와 권한을 함께 추측시킨다. 발생 시 unified_service_result.json에서 원인 상태를 확인

안전

Meeting·Review Service Router의 사람 승인과 external_write=false 위치

관측

unified_service_result.json에서 어떤 status·error_code를 보는가

unified_service_result.json 실행 증거

- 01 — 설명: Meeting·Review Service Router이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k router`
- 03 — 증거: unified_service_result.json과 정상·실패 test 결과가 남아 있다.

다음 차시: LangSmith Trace 구조

DAY 5 · 2차시

09:50-10:40 (쉬는 시간 11:30-12:00)

LangSmith Trace 구조

Project · Trace · trace.json 또는 LangSmith run

진행

LangSmith Trace 구조
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
src/workflow_service.py

확인할 결과

trace.json 또는 LangSmith run

2차시 입력과 결과

입력	————	unified_service_result.json
이번 차시	————	LangSmith Trace 구조
결과	————	trace.json 또는 LangSmith run
다음 연결	————	운영 Metadata와 Monitoring

LangSmith Trace 구조 필요성

최종 JSON만 저장하면 느린 node와 실패 원인을 다시 찾기 어렵다.

request start에서 trace.json 또는 LangSmith run까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: trace.json 또는 LangSmith run

LangSmith Trace 구조 학습 결과

- 01 — project.trace.run.thread의 관계를 이해하고 첫 trace를 남긴다.
- 02 — raw transcript.token.고객 정보를 trace input에 그대로 보낸다. 왜 위험한지 설명한다.
- 03 — trace.json 또는 LangSmith run에서 정상과 실패 상태를 직접 확인한다.

LangSmith Trace 구조 용어

용어	이 수업에서 쓰는 뜻
Project	관련 실행을 모으는 공간
Trace	한 요청의 전체 실행 계보
Run	node·LLM·tool 한 번의 실행
Thread	대화·재개 실행의 연속성

trace.json 또는 LangSmith run 입출력

INPUT

src/observability_lab.py

PROCESS

request start → span 시작 → 입력 최소화

OUTPUT

trace.json 또는 LangSmith run

VERIFY

trace에 span 계층과 READY/HOLD 결정이 남는다.
local_only나 key 없음은 upload하지 않고 error code를 반환한다.

LangSmith Trace 구조 실행 흐름

01

request start

02

span 시작

03

입력 최소화

04

status·latency

05

trace close

마지막 판단은 trace.json 또는 LangSmith run에서 사람이 확인합니다.

trace.json 또는 LangSmith run 정상·실패 계약

정상	trace에 span 계층과 READY/HOLD 결정이 남는다.
경계	local_only나 key 없음은 upload하지 않고 error code를 반환한다.
외부 쓰기	LangSmith Trace 구조: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·trace.json 또는 LangSmith run

LangSmith Trace 구조 파일 지도

입력·fixture

src/observability_lab.py

구현 코드

src/workflow_service.py

검증·test

output/day1-workflow/trace.json

따라하기 notebook

.env.sample

LangSmith Trace 구조 개념·코드

Project

관련 실행을 모으는 공간

```
src/observability_lab.py
```

Run

node·LLM·tool 한 번의 실행

```
output/day1-workflow/trace.json
```

Trace

한 요청의 전체 실행 계보

```
src/workflow_service.py
```

Thread

대화·재개 실행의 연속성

```
.env.sample
```


LangSmith Trace 구조 선택 기준

구분	역할	이번 차시의 판단 기준
Project	request start 단계의 입력 기준	결정론 test로 먼저 확인
Trace	span 시작 단계의 교체 경계	adapter 경계를 확인
Run	입력 최소화 단계의 운영 조건	비용·지연·권한을 확인
Thread	status·latency 단계의 판단 기록	사람 판단과 운영 기록을 확인

LangSmith Trace 구조 대표 실패

실패

**raw transcript-token·고객 정보를
trace input에 그대로 보낸다.**

사용자 영향

**raw transcript-token·고객 정보를 trace input
에 그대로 보낸다. 상태에서는 trace.json 또는
LangSmith run을 운영 판단에 사용할 수 없다.**

LangSmith Trace 구조 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

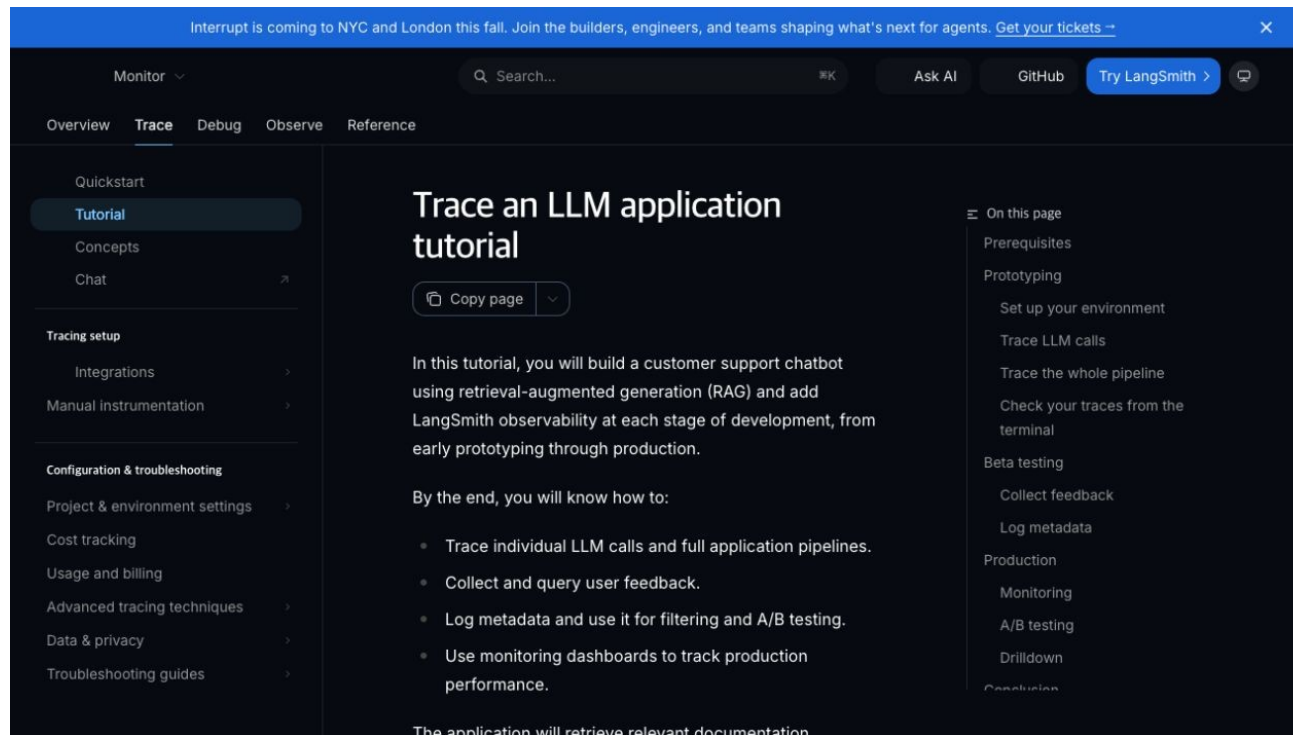
LangSmith Trace 구조 복구 경로

- 01 — 탐지: local_only나 key 없음은 upload하지 않고 error code를 반환한다.
- 02 — 중단: raw transcript·token·고객 정보를 trace input에 그대로 보낸다.
- 03 — 복구: local trace 기본, synthetic/deidentified만 명시 upload, raw content는 제외한다.
- 04 — 증거: trace.json 또는 LangSmith run과 test 결과를 함께 남긴다.

LangSmith Trace 구조 Codex 검증 루프

src/workflow_service.py의 책임을 찾고 trace.json 또는 LangSmith run 계약을 focused test로 고정합니다.

LangSmith Trace 구조
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



LangSmith Trace 구조 Codex 작업 명세

목표

새 node에 latency·status metadata를 추가하되 PII payload를 넣지 않는다.

허용 경로

src/workflow_service.py
output/day1-workflow/trace.json

완료 조건

trace에 span 계층과 READY/HOLD 결정이 남는다.
local_only나 key 없음은 upload하지 않고 error code를 반환한다.

금지

LangSmith Trace 구조: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

LangSmith Trace 구조 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | LangSmith Trace 구조 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | trace에 span 계층과 READY/HOLD 결정이 남는다. |
| 03 | Claude review | —— | trace.json 또는 LangSmith run 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | LangSmith Trace 구조의 두 LLM 의견과 test는 자동 승인이 아니다 |

LangSmith Trace 구조 정상 시연

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `src/workflow_service.py`
- 03 — 실행: `python -m src.workflow_service --decision approve --out output/day5-trace`
- 04 — 성공 신호: trace에 span 계층과 READY/HOLD 결정이 남는다.

LangSmith Trace 구조 실행 명령

```
$ python -m src.workflow_service --decision approve --out output/day5-trace
```

실행 전 확인

LangSmith Trace 구조: repository root · Python 3.12 · .env 미출력

02_trace.json 실행 결과

```
{
  "run_name": "day5-agent-operations",
  "started_at": "2026-08-24T15:57:41.224069+00:00",
  "finished_at": "2026-08-24T15:57:41.224124+00:00",
  "metadata": {
    "data_classification": "synthetic",
    "external_write": false
  },
  "spans": [
    {
      "name": "route_meeting",
      "status": "SUCCESS",
      "input_keys": [
        "input_kind"
      ],
      "latency_ms": 0
    },
    {
      "name": "route_code_review",
      "status": "SUCCESS",
      "input_keys": [
        "input_kind"
      ],
      "latency_ms": 0
    }
  ]
}
```

LangSmith Trace 구조 실패·복구 시연

재현

**local_only나 key 없음은 upload하지 않고
error code를 반환한다.**

복구

**local trace 기본,
synthetic/deidentified만 명시 upload,
raw content는 제외한다.**

확인: local_only나 key 없음은 upload하지 않고 error code를 반환한다.

LangSmith Trace 구조 소프트웨어 제작

열 파일

src/workflow_service.py
output/day1-workflow/trace.json

작업 범위

LangSmith Trace 구조 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

trace에 span 계층과 READY/HOLD 결정이 남는다.
trace.json 또는 LangSmith run 저장

LangSmith Trace 구조 Notebook 셀

materials/day5/day5_service_lab.ipynb · 2차시

```
from src.observability_lab import LocalTraceRecorder

trace_recorder = LocalTraceRecorder(
    run_name="day5-agent-operations",
    metadata={"data_classification": "synthetic", "external_write": False},
)
with trace_recorder.span("route_meeting", inputs={"input_kind": "meeting_transcript"}):
    assert meeting["status"] == "SUCCESS"
with trace_recorder.span("route_code_review", inputs={"input_kind": "code_diff"}):
    assert review["status"] == "SUCCESS"
trace = trace_recorder.to_dict()
trace["langsmith_upload"] = {"requested": False, "reason": "LOCAL_FIRST"}
save_json("02_trace.json", trace)
[(span["name"], span["status"]) for span in trace["spans"]]
```

LangSmith Trace 구조 Terminal 실행

```
$ python -m src.workflow_service --decision approve --out output/day5-trace
```

저장 위치

`output/course-labs/day5/02_trace.json`

trace에 span 계층과 READY/HOLD 결정이 남는다.
실패 시 trace.json 또는 LangSmith run의 status·error_code를 먼저 확인

LangSmith Trace 구조 실패 증거

local_only나 key 없음은 upload하지 않고 error code를 반환한다.

```
result_file = "output/course-labs/day5/02_trace.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

trace.json 또는 LangSmith run이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

LangSmith Trace 구조 정상 Case

NORMAL

trace에 span 계층과 READY/HOLD 결정이 남는다.

- 01 — src/observability_lab.py 입력과 command가 기록됐는가
- 02 — trace.json 또는 LangSmith run schema가 validate되는가
- 03 — LangSmith Trace 구조 focused test가 실제로 통과했는가

LangSmith Trace 구조 경계 Case

BOUNDARY

local_only나 key 없음은 upload하지 않고 error code를 반환한다.

- 01 — LangSmith Trace 구조: raw traceback 대신 stable error_code가 남는가
- 02 — trace.json 또는 LangSmith run: 외부 쓰기·자동 메일이 false인가
- 03 — local_only나 key 없음은 upload하지 않고 error code를 반환한다. 재현이 가능한가

LangSmith Trace 구조 현업 적용

장애 문의에서 request ID로 어느 node가 실패했는지 조회

- 01 — LangSmith Trace 구조은 합성·비식별 sample로 먼저 실행
- 02 — 장애 문의에서 request ID로 어느 node가 실패했는지 조회의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — trace.json 또는 LangSmith run 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — trace.json 또는 LangSmith run을 운영 검토 자료로 사용

LangSmith Trace 구조 포트폴리오 적용

LangSmith Trace 구조 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — raw transcript·token·고객 정보를 trace input에 그대로 보낸다.의 사용자 영향을 한 문장으로 설명
- 02 — LangSmith Trace 구조 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 새 node에 latency·status metadata를 추가하되 PII payload를 넣지 않는다.의 diff와 직접 검토한 test 증거를 구분
- 04 — trace.json 또는 LangSmith run과 README 재현 절차를 제출

LangSmith Trace 구조 운영 점검

품질

무엇을 trace에 span 계층과 READY/HOLD 결정이 남는다.로 정의하는가

복구

raw transcript·token·고객 정보를 trace input에 그대로 보낸다. 발생 시 trace.json 또는 LangSmith run에서 원인 상태를 확인

안전

LangSmith Trace 구조의 사람 승인과 external_write=false 위치

관측

trace.json 또는 LangSmith run에서 어떤 status·error_code를 보는가

trace.json 또는 LangSmith run 실행 증거

- 01 — 설명: LangSmith Trace 구조이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.workflow_service --decision approve --out output/day5-trace`
- 03 — 증거: trace.json 또는 LangSmith run과 정상·실패 test 결과가 남아 있다.

다음 차시: 운영 Metadata와 Monitoring

DAY 5 · 3차시

10:40-11:30 (쉬는 시간 11:30-12:00)

운영 Metadata와 Monitoring

Tag · Metadata · monitoring_view.md

진행

운영 Metadata와 Monitoring
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
output/day1-workflow/trace.json

확인할 결과

monitoring_view.md

3차시 입력과 결과

입력	————	trace.json 또는 LangSmith run
이번 차시	————	운영 Metadata와 Monitoring
결과	————	monitoring_view.md
다음 연결	————	Dataset Experiment와 Release Gate

운영 Metadata와 Monitoring 필요성

평균 latency 하나만 보면 특정 provider와 실패 유형의 꼬리가 숨겨진다.

tag version에서 monitoring_view.md까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: monitoring_view.md

운영 Metadata와 Monitoring 학습 결과

- 01 — status·error·latency·provider·cost metadata를 비교한다.
- 02 — 성공 trace만 보고 HOLD·reject·fallback을 제외한다. 왜 위험한지 설명한다.
- 03 — monitoring_view.md에서 정상과 실패 상태를 직접 확인한다.

운영 Metadata와 Monitoring 용어

용어	이 수업에서 쓰는 뜻
Tag	실행을 분류하는 짧은 label
Metadata	조회·비교용 구조화 값
Latency	요청부터 결과까지 걸린 시간
Cost	token·compute 사용 비용

monitoring_view.md 입출력

INPUT

src/observability_lab.py

OUTPUT

monitoring_view.md

PROCESS

tag version → provider metadata → node latency

VERIFY

provider_used와 node latency를 필터링할 수 있다.
오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.

운영 Metadata와 Monitoring 실행 흐름

01

tag version

02

provider metadata

03

node latency

04

error code

05

READY/HOLD filter

마지막 판단은 monitoring_view.md에서 사람이 확인합니다.

monitoring_view.md 정상·실패 계약

정상	—— provider_used와 node latency를 필터링할 수 있다.
경계	—— 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
외부 쓰기	—— 운영 Metadata와 Monitoring: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·monitoring_view.md

운영 Metadata와 Monitoring 파일 지도

입력·fixture

src/observability_lab.py

구현 코드

output/day1-workflow/trace.json

검증·test

materials/day5/day5_service_lab.ipynb

따라하기 notebook

AGENTS.md

운영 Metadata와 Monitoring 개념·코드

Tag

실행을 분류하는 짧은 label

```
src/observability_lab.py
```

Metadata

조회·비교용 구조화 값

```
output/day1-workflow/trace.json
```

Latency

요청부터 결과까지 걸린 시간

```
materials/day5/day5_service_lab.ipynb
```

Cost

token·compute 사용 비용

```
AGENTS.md
```

운영 Metadata와 Monitoring 선택 기준

구분	역할	이번 차시의 판단 기준
Tag	tag version 단계의 입력 기준	결정론 test로 먼저 확인
Metadata	provider metadata 단계의 교체 경계	adapter 경계를 확인
Latency	node latency 단계의 운영 조건	비용·지연·권한을 확인
Cost	error code 단계의 판단 기록	사람 판단과 운영 기록을 확인

운영 Metadata와 Monitoring 대표 실패

실패

성공 trace만 보고
HOLD·reject·fallback을 제외한다.

사용자 영향

성공 trace만 보고 HOLD·reject·fallback을
제외한다. 상태에서는 monitoring_view.md를
운영 판단에 사용할 수 없다.

운영 Metadata와 Monitoring 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

운영 Metadata와 Monitoring 복구 경로

- 01 — 탐지: 오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.
- 02 — 중단: 성공 trace만 보고 HOLD·reject·fallback을 제외한다.
- 03 — 복구: status와 fallback_reason을 같은 dashboard filter에 포함한다.
- 04 — 증거: monitoring_view.md과 test 결과를 함께 남긴다.

운영 Metadata와 Monitoring Codex 검증 루프

output/day1-workflow/
trace.json의 책임을 찾고
monitoring_view.md 계약을
focused test로 고정합니다.

운영 Metadata와 Monitoring
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

운영 Metadata와 Monitoring Codex 작업 명세

목표

추적 field를 늘리기 전에 운영 질문 한 개와 필요한 metadata만 정의한다.

허용 경로

output/day1-workflow/trace.json
materials/day5/day5_service_lab.ipynb

완료 조건

provider_used와 node latency를 필터링할 수 있다.
오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.

금지

운영 Metadata와 Monitoring: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

운영 Metadata와 Monitoring 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | 운영 Metadata와 Monitoring 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | provider_used와 node latency를 필터링할 수 있다. |
| 03 | Claude review | —— | monitoring_view.md 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 운영 Metadata와 Monitoring의 두 LLM 의견과 test는 자동 승인이 아니다 |

운영 Metadata와 Monitoring 정상 시연

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `output/day1-workflow/trace.json`
- 03 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 04 — 성공 신호: `provider_used`와 `node latency`를 필터링할 수 있다.

운영 Metadata와 Monitoring 실행 명령

```
$ python -m pytest -q tests/test_observability_workflow.py
```

실행 전 확인

운영 Metadata와 Monitoring: repository root · Python 3.12 · .env 미출력

03_monitoring_view.json 실행 결과

```
{
  "run_name": "day5-agent-operations",
  "status_counts": {
    "SUCCESS": 2,
    "ERROR": 0
  },
  "latency_ms": {
    "route_meeting": 0,
    "route_code_review": 0
  },
  "filters": [
    "provider_used",
    "error_code",
    "fallback_reason",
    "decision"
  ],
  "raw_content_uploaded": false
}
```

운영 Metadata와 Monitoring 실패·복구 시연

재현

오류 원문 대신 `stable error_code`와 `type`만 외부 `trace`에 남는다.

복구

`status`와 `fallback_reason`을 같은 `dashboard filter`에 포함한다.

확인: 오류 원문 대신 `stable error_code`와 `type`만 외부 `trace`에 남는다.

운영 Metadata와 Monitoring 소프트웨어 제작

열 파일

output/day1-workflow/trace.json
materials/day5/day5_service_lab.ipynb

작업 범위

운영 Metadata와 Monitoring 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

provider_used와 node latency를 필터링할 수 있다.
monitoring_view.md 저장

운영 Metadata와 Monitoring Notebook 셀

materials/day5/day5_service_lab.ipynb · 3차시

```
monitoring_view = {
    "run_name": trace["run_name"],
    "status_counts": {
        "SUCCESS": sum(span["status"] == "SUCCESS" for span in trace["spans"]),
        "ERROR": sum(span["status"] == "ERROR" for span in trace["spans"]),
    },
    "latency_ms": {span["name"]: span["latency_ms"] for span in trace["spans"]},
    "filters": ["provider_used", "error_code", "fallback_reason", "decision"],
    "raw_content_uploaded": False,
}
save_json("03_monitoring_view.json", monitoring_view)
monitoring_view
```

운영 Metadata와 Monitoring Terminal 실행

```
$ python -m pytest -q tests/test_observability_workflow.py
```

저장 위치

```
output/course-labs/day5/03_monitoring_view.json
```

provider_used와 node latency를 필터링할 수 있다.
실패 시 monitoring_view.md의 status·error_code를 먼저 확인

운영 Metadata와 Monitoring 실패 증거

오류 원문 대신 stable error_code와 type만 외부 trace에 남는다.

```
result_file = "output/course-labs/day5/03_monitoring_view.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

monitoring_view.md이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

운영 Metadata와 Monitoring 정상 Case

NORMAL

provider_used와 node latency를 필터링할 수 있다.

- 01 — src/observability_lab.py 입력과 command가 기록됐는가
- 02 — monitoring_view.md schema가 validate되는가
- 03 — 운영 Metadata와 Monitoring focused test가 실제로 통과했는가

운영 Metadata와 Monitoring 경계 Case

BOUNDARY

오류 원문 대신 `stable error_code`와 `type`만 외부 trace에 남는다.

- 01 — 운영 Metadata와 Monitoring: raw traceback 대신 `stable error_code`가 남는가
- 02 — `monitoring_view.md`: 외부 쓰기 자동 메일이 `false`인가
- 03 — 오류 원문 대신 `stable error_code`와 `type`만 외부 trace에 남는다. 재현이 가능한가

운영 Metadata와 Monitoring 현업 적용

모델 교체 후 latency·fallback·HOLD 비율 비교

- 01 — 운영 Metadata와 Monitoring은 합성·비식별 sample로 먼저 실행
- 02 — 모델 교체 후 latency·fallback·HOLD 비율 비교의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — monitoring_view.md 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — monitoring_view.md를 운영 검토 자료로 사용

운영 Metadata와 Monitoring 포트폴리오 적용

운영 Metadata와 Monitoring · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 성공 trace만 보고 HOLD·reject·fallback을 제외한다.의 사용자 영향을 한 문장으로 설명
- 02 ——— 운영 Metadata와 Monitoring 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 추적 field를 늘리기 전에 운영 질문 한 개와 필요한 metadata만 정의한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— monitoring_view.md과 README 재현 절차를 제출

운영 Metadata와 Monitoring 운영 점검

품질

무엇을 provider_used와 node latency를 필터링할 수 있다.로 정의하는가

복구

성공 trace만 보고 HOLD·reject·fallback을 제외한다.
발생 시 monitoring_view.md에서 원인 상태를 확인

안전

운영 Metadata와 Monitoring의 사람 승인과
external_write=false 위치

관측

monitoring_view.md에서 어떤 status·error_code를
보는가

monitoring_view.md 실행 증거

- 01 — 설명: 운영 Metadata와 Monitoring이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 03 — 증거: monitoring_view.md과 정상·실패 test 결과가 남아 있다.

다음 차시: Dataset Experiment와 Release Gate

DAY 5 · 4차시

12:00-12:50 (쉬는 시간 13:40-14:00)

Dataset Experiment와 Release Gate

Dataset · Experiment · experiment_result.json

진행

Dataset Experiment와 Release Gate
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/eval_service.py
data/day5_eval/
golden_review_findings.json

확인할 결과

experiment_result.json

4차시 입력과 결과

입력	————	monitoring_view.md
이번 차시	————	Dataset Experiment와 Release Gate
결과	————	experiment_result.json
다음 연결	————	Human Feedback과 Annotation

Dataset Experiment와 Release Gate 필요성

데모 입력이 매번 달라지면 개선인지 운인지 구분할 수 없다.

golden load에서 experiment_result.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: experiment_result.json

Dataset Experiment와 Release Gate 학습 결과

- 01 — golden review case를 code evaluator로 실행한다.
- 02 — LLM judge 한 개 점수를 객관적 정답처럼 사용한다. 왜 위험한지 설명한다.
- 03 — experiment_result.json에서 정상과 실패 상태를 직접 확인한다.

Dataset Experiment와 Release Gate 용어

용어	이 수업에서 쓰는 뜻
Dataset	반복 평가할 입력·정답 묶음
Experiment	특정 버전을 dataset에 실행
Code evaluator	규칙으로 자동 채점
Candidate	새로 비교할 구현

experiment_result.json 입출력

INPUT

src/course_services/eval_service.py

OUTPUT

experiment_result.json

PROCESS

golden load → baseline run → candidate run

VERIFY

fixture precision·recall·F1이 1.0이다.
precision 또는 recall 0.8 미만은 HOLD다.

Dataset Experiment와 Release Gate 실행 흐름

01

golden load

02

baseline run

03

candidate run

04

metric compare

05

release gate

마지막 판단은 `experiment_result.json`에서 사람이 확인합니다.

experiment_result.json 정상·실패 계약

정상	—— fixture precision·recall·F1이 1.0이다.
경계	—— precision 또는 recall 0.8 미만은 HOLD다.
외부 쓰기	—— Dataset Experiment와 Release Gate: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·experiment_result.json

Dataset Experiment와 Release Gate 파일 지도

입력·fixture

`src/course_services/eval_service.py`

구현 코드

`data/day5_eval/golden_review_findings.json`

검증·test

`tests/test_course_services.py`

따라하기 notebook

`materials/day5/day5_service_lab.ipynb`

Dataset Experiment와 Release Gate 개념·코드

Dataset

반복 평가할 입력·정답 묶음

```
src/course_services/eval_service.py
```

Experiment

특정 버전을 dataset에 실행

```
data/day5_eval/golden_review_findings.json
```

Code evaluator

규칙으로 자동 채점

```
tests/test_course_services.py
```

Candidate

새로 비교할 구현

```
materials/day5/day5_service_lab.ipynb
```

Dataset Experiment와 Release Gate 선택 기준

구분	역할	이번 차시의 판단 기준
Dataset	golden load 단계의 입력 기준	결정론 test로 먼저 확인
Experiment	baseline run 단계의 교체 경계	adapter 경계를 확인
Code evaluator	candidate run 단계의 운영 조건	비용·지연·권한을 확인
Candidate	metric compare 단계의 판단 기록	사람 판단과 운영 기록을 확인

Dataset Experiment와 Release Gate 대표 실패

실패

LLM judge 한 개 점수를 객관적
정답처럼 사용한다.

사용자 영향

LLM judge 한 개 점수를 객관적 정답처럼
사용한다. 상태에서는 experiment_result.json
을 운영 판단에 사용할 수 없다.

Dataset Experiment와 Release Gate 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

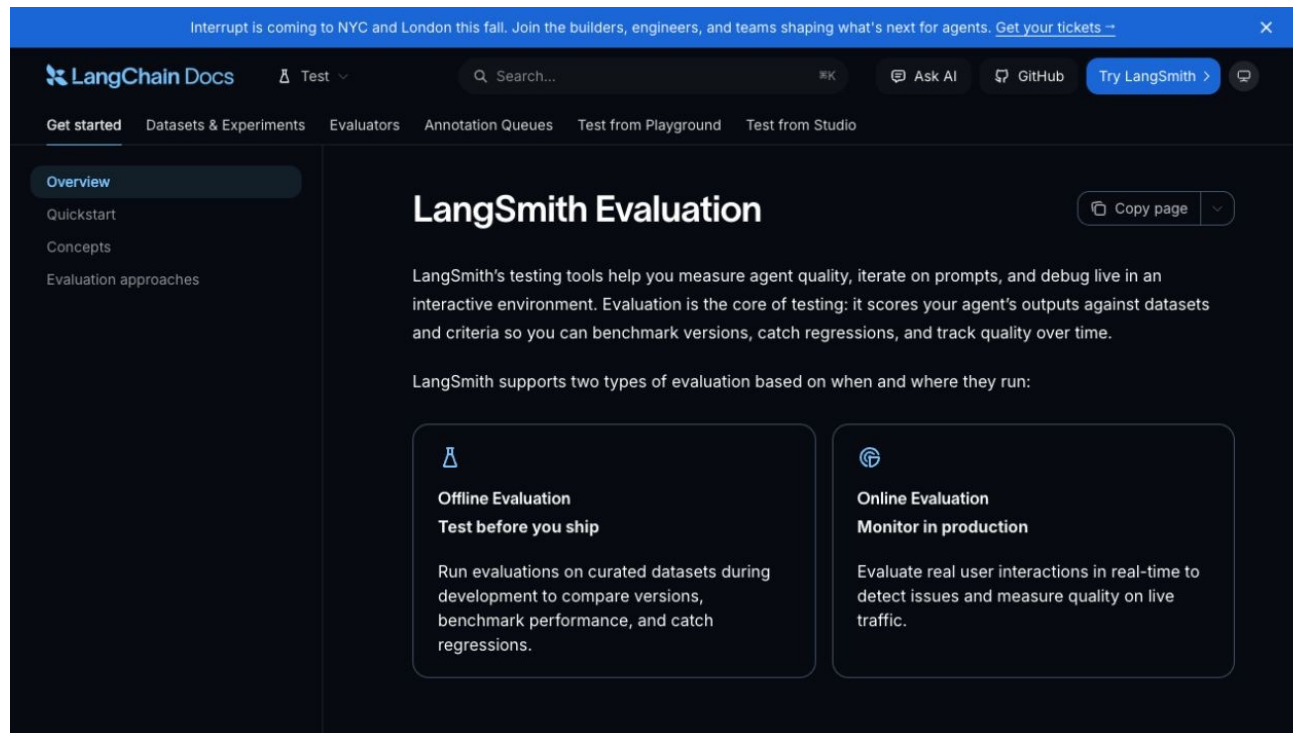
Dataset Experiment와 Release Gate 복구 경로

- 01 — 탐지: precision 또는 recall 0.8 미만은 HOLD다.
- 02 — 중단: LLM judge 한 개 점수를 객관적 정답처럼 사용한다.
- 03 — 복구: schema·line mapping·policy는 code evaluator, 의미 품질은 human sample로 나눈다.
- 04 — 증거: experiment_result.json과 test 결과를 함께 남긴다.

Dataset Experiment와 Release Gate Codex 검증 루프

data/day5_eval/
golden_review_findings.json의
책임을 찾고
experiment_result.json 계약을
focused test로 고정합니다.

Dataset Experiment와 Release Gate
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Dataset Experiment와 Release Gate Codex 작업 명세

목표

새 평가 지표에 empty set과 threshold boundary test를 추가한다.

허용 경로

data/day5_eval/golden_review_findings.json
tests/test_course_services.py

완료 조건

fixture precision·recall·F1이 1.0이다.
precision 또는 recall 0.8 미만은 HOLD다.

금지

Dataset Experiment와 Release Gate: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Dataset Experiment와 Release Gate 독립 리뷰

- 01 **Codex diff** — Dataset Experiment와 Release Gate 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — fixture precision·recall·F1이 1.00이다.
- 03 **Claude review** — experiment_result.json 변경 라인·누락 경계를 별도 session에서 검토
- 04 **Human merge** — Dataset Experiment와 Release Gate의 두 LLM 의견과 test는 자동 승인이 아니다

Dataset Experiment와 Release Gate 정상 시연

- 01 — 열기: `src/course_services/eval_service.py`
- 02 — 구현 확인: `data/day5_eval/golden_review_findings.json`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 04 — 성공 신호: fixture precision·recall·F1이 1.0이다.

Dataset Experiment와 Release Gate 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

실행 전 확인

Dataset Experiment와 Release Gate: repository root · Python 3.12 · .env 미출력

04_experiment_result.json 실행 결과

```
{
  "metrics": {
    "true_positive": 3,
    "false_positive": 0,
    "false_negative": 0,
    "precision": 1,
    "recall": 1,
    "f1": 1
  },
  "normal": {
    "decision": "READY",
    "reasons": [],
    "human_release_required": true
  },
  "boundary": {
    "decision": "HOLD",
    "reasons": [
      "RECALL_BELOW_0_8",
      "PRECISION_BELOW_0_8",
      "SAFETY_CHECK_FAILED",
      "LATENCY_BUDGET_EXCEEDED"
    ],
    "human_release_required": true
  }
}
```

Dataset Experiment와 Release Gate 실패·복구 시연

재현

precision 또는 recall 0.8 미만은 HOLD
다.

복구

schema·line mapping·policy는 code
evaluator, 의미 품질은 human sample
로 나눈다.

확인: precision 또는 recall 0.8 미만은 HOLD다.

Dataset Experiment와 Release Gate 소프트웨어 제작

열 파일 `data/day5_eval/golden_review_findings.json`
`tests/test_course_services.py`

작업 범위 Dataset Experiment와 Release Gate 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 fixture precision·recall·F1이 1.0이다.
`experiment_result.json` 저장

Dataset Experiment와 Release Gate Notebook 셀

materials/day5/day5_service_lab.ipynb · 4차시

```
from src.course_services.eval_service import evaluate_review_findings, release_gate

expected = json.loads((ROOT / "data/day5_eval/golden_review_findings.json").read_text(encoding="utf-8"))
experiment_metrics = evaluate_review_findings(review["result"]["findings"], expected)
ready_gate = release_gate(review_metrics=experiment_metrics, safety_passed=True, latency_seconds=0.2)
hold_gate = release_gate(review_metrics={"precision": 0.7, "recall": 0.6}, safety_passed=False, latency_seconds=42.0)
assert ready_gate["decision"] == "READY" and hold_gate["decision"] == "HOLD"
save_json("04_experiment_result.json", {"metrics": experiment_metrics, "normal": ready_gate, "boundary": hold_gate})
{"metrics": experiment_metrics, "normal": ready_gate["decision"], "boundary": hold_gate["decision"]}
```


Dataset Experiment와 Release Gate Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

저장 위치

```
output/course-labs/day5/04_experiment_result.json
```

fixture precision·recall·F1이 1.0이다.

실패 시 experiment_result.json의 status·error_code를 먼저 확인

Dataset Experiment와 Release Gate 실패 증거

precision 또는 recall 0.8 미만은 HOLD다.

```
result_file = "output/course-labs/day5/04_experiment_result.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

experiment_result.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Dataset Experiment와 Release Gate 정상 Case

NORMAL

fixture precision·recall·F1이 1.0이다.

- 01 — src/course_services/eval_service.py 입력과 command가 기록됐는가
- 02 — experiment_result.json schema가 validate되는가
- 03 — Dataset Experiment와 Release Gate focused test가 실제로 통과했는가

Dataset Experiment와 Release Gate 경계 Case

BOUNDARY

precision 또는 recall 0.8 미만은 HOLD다.

- 01 — Dataset Experiment와 Release Gate: raw traceback 대신 stable error_code가 남는가
- 02 — experiment_result.json: 외부 쓰기·자동 메일이 false인가
- 03 — precision 또는 recall 0.8 미만은 HOLD다. 재현이 가능한가

Dataset Experiment와 Release Gate 현업 적용

릴리스 후보가 기존 version보다 실제로 나아졌는지 비교

- 01 — Dataset Experiment와 Release Gate은 합성·비식별 sample로 먼저 실행
- 02 — 릴리스 후보가 기존 version보다 실제로 나아졌는지 비교의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — experiment_result.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — experiment_result.json을 운영 검토 자료로 사용

Dataset Experiment와 Release Gate 포트폴리오 적용

Dataset Experiment와 Release Gate · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — LLM judge 한 개 점수를 객관적 정답처럼 사용한다.의 사용자 영향을 한 문장으로 설명
- 02 — Dataset Experiment와 Release Gate 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 새 평가 지표에 empty set과 threshold boundary test를 추가한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — experiment_result.json과 README 재현 절차를 제출

Dataset Experiment와 Release Gate 운영 점검

품질

무엇을 fixture precision·recall·F1이 1.0이다.로 정의하는가

복구

LLM judge 한 개 점수를 객관적 정답처럼 사용한다. 발생 시 experiment_result.json에서 원인 상태를 확인

안전

Dataset Experiment와 Release Gate의 사람 승인과 external_write=false 위치

관측

experiment_result.json에서 어떤 status·error_code를 보는가

experiment_result.json 실행 증거

- 01 — 설명: Dataset Experiment와 Release Gate이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 03 — 증거: experiment_result.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Human Feedback과 Annotation

DAY 5 · 5차시

12:50-13:40 (쉬는 시간 13:40-14:00)

Human Feedback과 Annotation

Annotation · Feedback · human_feedback.jsonl

진행

Human Feedback과 Annotation
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/meeting_agent_workflow.py
src/langgraph_lab.py

확인할 결과

human_feedback.jsonl

5차시 입력과 결과

입력	————	experiment_result.json
이번 차시	————	Human Feedback과 Annotation
결과	————	human_feedback.jsonl
다음 연결	————	PII·Retention·Incident 운영

Human Feedback과 Annotation 필요성

단순 thumbs-up은 어떤 오류를 고쳐야 하는지 알려주지 못한다.

HOLD sample에서 human_feedback.jsonl까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: human_feedback.jsonl

Human Feedback과 Annotation 학습 결과

- 01 — approve·edit·reject와 수정 이유를 label로 남긴다.
- 02 — 사람 수정 내용을 곧바로 training data로 재사용한다. 왜 위험한지 설명한다.
- 03 — human_feedback.jsonl에서 정상과 실패 상태를 직접 확인한다.

Human Feedback과 Annotation 용어

용어	이 수업에서 쓰는 뜻
Annotation	사람이 결과에 붙인 label
Feedback	실행 품질에 대한 평가
Queue	검토가 필요한 실행 목록
Reviewer agreement	검토자 판단 일치 정도

human_feedback.jsonl 입출력

INPUT

src/meeting_agent_workflow.py

OUTPUT

human_feedback.jsonl

PROCESS

HOLD sample → 근거 표시 → 사람 결정

VERIFY

approve·edit·reject가 reviewer·reason과 저장된다.
PII가 있는 raw content는 annotation export에서 제외된다.

Human Feedback과 Annotation 실행 흐름

01

HOLD sample

02

근거 표시

03

사람 결정

04

reason label

05

dataset 후보

마지막 판단은 human_feedback.jsonl에서 사람이 확인합니다.

human_feedback.jsonl 정상·실패 계약

정상	approve·edit·reject가 reviewer·reason과 저장된다.
경계	PII가 있는 raw content는 annotation export에서 제외된다.
외부 쓰기	Human Feedback과 Annotation: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·human_feedback.jsonl

Human Feedback과 Annotation 파일 지도

입력·fixture

src/meeting_agent_workflow.py

구현 코드

src/langgraph_lab.py

검증·test

web-demo/app.js

따라하기 notebook

materials/day5/day5_service_lab.ipynb

Human Feedback과 Annotation 개념·코드

Annotation

사람이 결과에 붙인 label

```
src/meeting_agent_workflow.py
```

Feedback

실행 품질에 대한 평가

```
src/langgraph_lab.py
```

Queue

검토가 필요한 실행 목록

```
web-demo/app.js
```

Reviewer agreement

검토자 판단 일치 정도

```
materials/day5/day5_service_lab.ipynb
```

Human Feedback과 Annotation 선택 기준

구분	역할	이번 차시의 판단 기준
Annotation	HOLD sample 단계의 입력 기준	결정론 test로 먼저 확인
Feedback	근거 표시 단계의 교체 경계	adapter 경계를 확인
Queue	사람 결정 단계의 운영 조건	비용·지연·권한을 확인
Reviewer agreement	reason label 단계의 판단 기록	사람 판단과 운영 기록을 확인

Human Feedback과 Annotation 대표 실패

실패

사람 수정 내용을 곧바로 training data로 재사용한다.

사용자 영향

사람 수정 내용을 곧바로 training data로 재사용한다. 상태에서는 human_feedback.jsonl을 운영 판단에 사용할 수 없다.

Human Feedback과 Annotation 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Human Feedback과 Annotation 복구 경로

- 01 — 탐지: PII가 있는 raw content는 annotation export에서 제외된다.
- 02 — 중단: 사람 수정 내용을 곧바로 training data로 재사용한다.
- 03 — 복구: 동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.
- 04 — 증거: human_feedback.jsonl과 test 결과를 함께 남긴다.

Human Feedback과 Annotation Codex 검증 루프

src/langgraph_lab.py의 책임을
찾고 human_feedback.jsonl
계약을 focused test로 고정합니다.

Human Feedback과 Annotation
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

WORKFLOW LAB

Pipeline Review Result

02 LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

● REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리	서연 · 2026-08-27
근거 s12, s18	
응답 JSON Schema와 실패 테스트 작성	준호 · 2026-08-28
근거 s27, s31	

interrupt_payload.json read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
    }
  ]
}
```

Human Feedback과 Annotation Codex 작업 명세

목표

feedback schema와 allowed label을 구현하고 unknown label을 차단한다.

허용 경로

src/langgraph_lab.py
web-demo/app.js

완료 조건

approve·edit·reject가 reviewer·reason과 저장된다.
PII가 있는 raw content는 annotation export에서 제외된다.

금지

Human Feedback과 Annotation: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Human Feedback과 Annotation 독립 리뷰

- | | | | |
|----|----------------------|------|--|
| 01 | Codex diff | ———— | Human Feedback과 Annotation 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | approve·edit·reject가 reviewer·reason과 저장된다. |
| 03 | Claude review | ———— | human_feedback.jsonl 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | Human Feedback과 Annotation의 두 LLM 의견과 test는 자동 승인이 아니다 |

Human Feedback과 Annotation 정상 시연

- 01 — 열기: `src/meeting_agent_workflow.py`
- 02 — 구현 확인: `src/langgraph_lab.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve·edit·reject`가 `reviewer·reason`과 저장된다.

Human Feedback과 Annotation 실행 명령

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

Human Feedback과 Annotation: repository root · Python 3.12 · .env 미출력

05_human_feedback.jsonl 실행 결과

```
{"run_id": "synthetic-001", "decision": "approve", "reason": "EVIDENCE_CONFIRMED", "reviewer": "human"}  
{"run_id": "synthetic-002", "decision": "edit", "reason": "OWNER_CORRECTED", "reviewer": "human"}  
{"run_id": "synthetic-003", "decision": "reject", "reason": "UNKNOWN_EVIDENCE", "reviewer": "human"}
```

Human Feedback과 Annotation 실패·복구 시연

재현

PII가 있는 raw content는 annotation export에서 제외된다.

복구

동의·비식별·품질 검토 후 별도 dataset 승인을 거친다.

확인: PII가 있는 raw content는 annotation export에서 제외된다.

Human Feedback과 Annotation 소프트웨어 제작

열 파일 `src/langgraph_lab.py`
`web-demo/app.js`

작업 범위 Human Feedback과 Annotation 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 `approve·edit·reject`가 `reviewer·reason`과 저장된다.
`human_feedback.jsonl` 저장

Human Feedback과 Annotation Notebook 셀

materials/day5/day5_service_lab.ipynb · 5차시

```
feedback = [  
    {"run_id": "synthetic-001", "decision": "approve", "reason": "EVIDENCE_CONFIRMED", "reviewer": "human"},  
    {"run_id": "synthetic-002", "decision": "edit", "reason": "OWNER_CORRECTED", "reviewer": "human"},  
    {"run_id": "synthetic-003", "decision": "reject", "reason": "UNKNOWN_EVIDENCE", "reviewer": "human"},  
]  
feedback_path = OUT / "05_human_feedback.jsonl"  
feedback_path.write_text(''.join(json.dumps(row, ensure_ascii=False) + "\n" for row in feedback), encoding="utf-8")  
print({"saved": str(feedback_path.relative_to(ROOT)), "rows": len(feedback)})  
feedback
```

Human Feedback과 Annotation Terminal 실행

```
$ python -m src.langgraph_lab --decision all
```

저장 위치

```
output/course-labs/day5/05_human_feedback.jsonl
```

approve·edit·reject가 reviewer·reason과 저장된다.
실패 시 human_feedback.jsonl의 status·error_code를 먼저 확인

Human Feedback과 Annotation 실패 증거

PII가 있는 raw content는 annotation export에서 제외된다.

```
result_file = "output/course-labs/day5/05_human_feedback.jsonl"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

human_feedback.jsonl이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Human Feedback과 Annotation 정상 Case

NORMAL

approve·edit·reject가 reviewer·reason과 저장된다.

- 01 — src/meeting_agent_workflow.py 입력과 command가 기록됐는가
- 02 — human_feedback.jsonl schema가 validate되는가
- 03 — Human Feedback과 Annotation focused test가 실제로 통과했는가

Human Feedback과 Annotation 경계 Case

BOUNDARY

PII가 있는 raw content는 annotation export에서 제외된다.

- 01 — Human Feedback과 Annotation: raw traceback 대신 stable error_code가 남는가
- 02 — human_feedback.jsonl: 외부 쓰기·자동 메일이 false인가
- 03 — PII가 있는 raw content는 annotation export에서 제외된다. 재현이 가능한가

Human Feedback과 Annotation 현업 적용

운영자 검토 결과를 다음 prompt·rule 개선 후보로 연결

- 01 — Human Feedback과 Annotation은 합성·비식별 sample로 먼저 실행
- 02 — 운영자 검토 결과를 다음 prompt·rule 개선 후보로 연결의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — human_feedback.jsonl 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — human_feedback.jsonl을 운영 검토 자료로 사용

Human Feedback과 Annotation 포트폴리오 적용

Human Feedback과 Annotation · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 사람 수정 내용을 곧바로 training data로 재사용한다.의 사용자 영향을 한 문장으로 설명
- 02 — Human Feedback과 Annotation 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — feedback schema와 allowed label을 구현하고 unknown label을 차단한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — human_feedback.jsonl과 README 재현 절차를 제출

Human Feedback과 Annotation 운영 점검

품질

무엇을 approve·edit·reject가 reviewer·reason과 저장된다.로 정의하는가

복구

사람 수정 내용을 곧바로 training data로 재사용한다. 발생 시 human_feedback.jsonl에서 원인 상태를 확인

안전

Human Feedback과 Annotation의 사람 승인과 external_write=false 위치

관측

human_feedback.jsonl에서 어떤 status·error_code를 보는가

human_feedback.jsonl 실행 증거

- 01 — 설명: Human Feedback과 Annotation이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: human_feedback.jsonl과 정상·실패 test 결과가 남아 있다.

다음 차시: PII·Retention·Incident 운영

DAY 5 · 6차시

15:00-15:50 (쉬는 시간 17:30-18:00)

PII·Retention·Incident 운영

PII · Redaction · ops_checklist.md

진행

PII·Retention·Incident 운영
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/observability_lab.py
AGENTS.md

확인할 결과

ops_checklist.md

6차시 입력과 결과

입력	————	human_feedback.jsonl
이번 차시	————	PII·Retention·Incident 운영
결과	————	ops_checklist.md
다음 연결	————	Release Candidate와 Vercel Demo

PII-Retention-Incident 운영 필요성

회의·상담·코드에는 개인 정보와 회사 비밀이 동시에 들어갈 수 있다.

classify에서 ops_checklist.md까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: ops_checklist.md

PII-Retention-Incident 운영 학습 결과

- 01 — redaction·retention·access·incident checklist를 만든다.
- 02 — 편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다. 왜 위험한지 설명한다.
- 03 — ops_checklist.md에서 정상과 실패 상태를 직접 확인한다.

PII·Retention·Incident 운영 용어

용어	이 수업에서 쓰는 뜻
PII	개인을 식별할 수 있는 정보
Redaction	민감 정보를 가리거나 제거
Retention	데이터 보관 기간
Incident	보안·품질 장애 사건

ops_checklist.md 입출력

INPUT

src/observability_lab.py

OUTPUT

ops_checklist.md

PROCESS

classify → minimize → redact

VERIFY

synthetic·deidentified만 외부 upload 후보가 된다.
local_only와 secret 탐지는 upload를 차단한다.

PII·Retention·Incident 운영 실행 흐름

01

classify

02

minimize

03

redact

04

retain/delete

05

incident review

마지막 판단은 ops_checklist.md에서 사람이 확인합니다.

ops_checklist.md 정상·실패 계약

정상	——	synthetic·deidentified만 외부 upload 후보가 된다.
경계	——	local_only와 secret 탐지는 upload를 차단한다.
외부 쓰기	——	PII·Retention·Incident 운영: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·ops_checklist.md

PII·Retention·Incident 운영 파일 지도

입력·fixture

src/observability_lab.py

구현 코드

AGENTS.md

검증·test

.env.sample

따라하기 notebook

materials/day1/강사_회의음성_라이브데모_런북.md

PII·Retention·Incident 운영 개념·코드

PII

개인을 식별할 수 있는 정보

`src/observability_lab.py`

Redaction

민감 정보를 가리거나 제거

`AGENTS.md`

Retention

데이터 보관 기간

`.env.sample`

Incident

보안·품질 장애 사건

`materials/day1/강사_회의음성_라이브데모_런북.md`

PII·Retention·Incident 운영 선택 기준

구분	역할	이번 차시의 판단 기준
PII	classify 단계의 입력 기준	결정론 test로 먼저 확인
Redaction	minimize 단계의 교체 경계	adapter 경계를 확인
Retention	redact 단계의 운영 조건	비용·지연·권한을 확인
Incident	retain/delete 단계의 판단 기록	사람 판단과 운영 기록을 확인

PII·Retention·Incident 운영 대표 실패

실패

편의를 위해 원본 audio와 raw prompt
를 무기한 외부에 보관한다.

사용자 영향

편의를 위해 원본 audio와 raw prompt를 무기한
외부에 보관한다. 상태에서는 ops_checklist.md
을 운영 판단에 사용할 수 없다.

PII·Retention·Incident 운영 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

PII·Retention·Incident 운영 복구 경로

- 01 — 탐지: local_only와 secret 탐지는 upload를 차단한다.
- 02 — 중단: 편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다.
- 03 — 복구: 목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.
- 04 — 증거: ops_checklist.md과 test 결과를 함께 남긴다.

PII·Retention·Incident 운영 Codex 검증 루프

AGENTS.md의 책임을 찾고
ops_checklist.md 계약을
focused test로 고정합니다.

PII·Retention·Incident 운영
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

The screenshot shows the Google Meet Help Center interface. At the top, there's a search bar with the text "문제를 설명해 주세요." and a "로그인" button. Below the search bar is a navigation bar with links: "Google Meet 고객센터", "Google Meet 시작하기", "화상 회의 시작 또는 참여하기", "Gemini in Meet", "회의 중 작업", "회의 종료 후", "기능 사용 가능 여부", "Google Meet 설정", and "Google Meet에서 통화하기". The "Gemini in Meet" link is highlighted. The main content area displays the article "Google Meet의 자동 회의록". The article text includes a "중요:" section and a list of supported languages: 영어, 프랑스어, 독일어, 이탈리아어, 일본어, 한국어, 포르투갈어, and 스페인어. Below the list, there's a note about the feature being available for one language at a time and a link to "자동 회의록의 언어를 변경하는 방법 알아보기". On the right side, there's a "Help Center" sidebar with a list of related articles, including "Gemini 기반 Google Workspace 시작하기", "Google Meet의 Gemini로 배경 이미지 만들기", "Enhance your video & audio with Gemini in Google Meet", "Google Meet의 자동 회의록", "오프라인 회의에 자동 회의록 사용하기", "Meet에서 스마트 오디오 사용하기", "Google Meet의 Gemini에게 물어보기", "통역 기능에 관해 알아보기", "Calendar 및 Meet의 자동 회의록 설정", and "Workspace의 Gemini 대화 기록 찾기 및 과".

PII·Retention·Incident 운영 Codex 작업 명세

목표

trace scrubber가 token·email·전화번호를 제거하는 정상·실패 test를 만든다.

허용 경로

AGENTS.md
.env.sample

완료 조건

synthetic·deidentified만 외부 upload 후보가 된다.
local_only와 secret 탐지는 upload를 차단한다.

금지

PII·Retention·Incident 운영: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

PII·Retention·Incident 운영 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | PII·Retention·Incident 운영 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | synthetic·deidentified만 외부 upload 후보가 된다. |
| 03 | Claude review | —— | ops_checklist.md 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | PII·Retention·Incident 운영의 두 LLM 의견과 test는 자동 승인이 아니다 |

PII·Retention·Incident 운영 정상 시연

- 01 — 열기: `src/observability_lab.py`
- 02 — 구현 확인: `AGENTS.md`
- 03 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 04 — 성공 신호: `synthetic·deidentified`만 외부 upload 후보가 된다.

PII-Retention-Incident 운영 실행 명령

```
$ python -m pytest -q tests/test_observability_workflow.py
```

실행 전 확인

PII-Retention-Incident 운영: repository root · Python 3.12 · .env 미출력

06_ops_checklist.json 실행 결과

```
{
  "classification": "synthetic",
  "redaction": {
    "text": "담당자 [EMAIL_REDACTED] [PHONE_REDACTED] [TOKEN_REDACTED]",
    "redacted": true,
    "replacements": {
      "EMAIL_REDACTED": 1,
      "PHONE_REDACTED": 1,
      "TOKEN_REDACTED": 1
    }
  },
  "retention_days": 7,
  "raw_content_upload": false,
  "incident_owner_required": true
}
```

PII·Retention·Incident 운영 실패·복구 시연

재현

local_only와 secret 탐지는 upload를 차단한다.

복구

목적·동의·최소수집·보존기한·삭제 경로를 배포 전 결정한다.

확인: local_only와 secret 탐지는 upload를 차단한다.

PII·Retention·Incident 운영 소프트웨어 제작

열 파일	AGENTS.md .env.sample
작업 범위	PII·Retention·Incident 운영 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	synthetic·deidentified만 외부 upload 후보가 된다. ops_checklist.md 저장

PII-Retention-Incident 운영 Notebook 셀

materials/day5/day5_service_lab.ipynb · 6차시

```
from src.observability_lab import redact_observability_text

sample_sensitive = "담당자 test@example.com 010-1234-5678 sk-exampletoken123456789"
redaction = redact_observability_text(sample_sensitive)
ops_checklist = {
    "classification": "synthetic", "redaction": redaction, "retention_days": 7,
    "raw_content_upload": False, "incident_owner_required": True,
}
assert redaction["redacted"] is True
assert "test@example.com" not in redaction["text"]
save_json("06_ops_checklist.json", ops_checklist)
ops_checklist
```

PII·Retention·Incident 운영 Terminal 실행

```
$ python -m pytest -q tests/test_observability_workflow.py
```

저장 위치

```
output/course-labs/day5/06_ops_checklist.json
```

synthetic·deidentified만 외부 upload 후보가 된다.
실패 시 ops_checklist.md의 status·error_code를 먼저 확인

PII·Retention·Incident 운영 실패 증거

local_only와 secret 탐지는 upload를 차단한다.

```
result_file = "output/course-labs/day5/06_ops_checklist.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

ops_checklist.md이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

PII·Retention·Incident 운영 정상 Case

NORMAL

synthetic·deidentified만 외부 upload 후보가 된다.

- 01 — src/observability_lab.py 입력과 command가 기록됐는가
- 02 — ops_checklist.md schema가 validate되는가
- 03 — PII·Retention·Incident 운영 focused test가 실제로 통과했는가

PII·Retention·Incident 운영 경계 Case

BOUNDARY

local_only와 secret 탐지는 upload를 차단한다.

- 01 — PII·Retention·Incident 운영: raw traceback 대신 stable error_code가 남는가
- 02 — ops_checklist.md: 외부 쓰기·자동 메일이 false인가
- 03 — local_only와 secret 탐지는 upload를 차단한다. 재현이 가능한가

PII·Retention·Incident 운영 현업 적용

교육 데모에서 시작해 실제 고객 데이터로 넘어가기 위한 readiness gate

- 01 — PII·Retention·Incident 운영은 합성·비식별 sample로 먼저 실행
- 02 — 교육 데모에서 시작해 실제 고객 데이터로 넘어가기 위한 readiness gate의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — ops_checklist.md 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — ops_checklist.md를 운영 검토 자료로 사용

PII·Retention·Incident 운영 포트폴리오 적용

PII·Retention·Incident 운영 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다.의 사용자 영향을 한 문장으로 설명
- 02 — PII·Retention·Incident 운영 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — trace scrubber가 token·email·전화번호를 제거하는 정상·실패 test를 만든다.의 diff와 직접 검토한 test 증거를 구분
- 04 — ops_checklist.md과 README 재현 절차를 제출

PII·Retention·Incident 운영 운영 점검

품질

무엇을 synthetic·deidentified만 외부 upload 후보가 된다.로 정의하는가

복구

편의를 위해 원본 audio와 raw prompt를 무기한 외부에 보관한다. 발생 시 ops_checklist.md에서 원인 상태를 확인

안전

PII·Retention·Incident 운영의 사람 승인과 external_write=false 위치

관측

ops_checklist.md에서 어떤 status·error_code를 보는가

ops_checklist.md 실행 증거

- 01 — 설명: PII·Retention·Incident 운영이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_observability_workflow.py`
- 03 — 증거: ops_checklist.md과 정상·실패 test 결과가 남아 있다.

다음 차시: Release Candidate와 Vercel Demo

DAY 5 · 7차시

15:50-16:40 (쉬는 시간 17:30-18:00)

Release Candidate와 Vercel Demo

Release candidate · Smoke test · release_candidate

진행

Release Candidate와 Vercel Demo
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

README.md
web-demo/

확인할 결과

release_candidate

7차시 입력과 결과

입력



ops_checklist.md

이번 차시



Release Candidate와 Vercel Demo

결과



release_candidate

다음 연결



최종 Demo와 Scorecard

Release Candidate와 Vercel Demo 필요성

코드는 최신인데 화면과 문서가 이전 schema면 사용자는 첫 실행에서 막힌다.

focused tests에서 release_candidate까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: release_candidate

Release Candidate와 Vercel Demo 학습 결과

- 01 — test·README·sample data·Vercel demo를 release candidate로 묶는다.
- 02 — Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다. 왜 위험한지 설명한다.
- 03 — release_candidate에서 정상과 실패 상태를 직접 확인한다.

Release Candidate와 Vercel Demo 용어

용어	이 수업에서 쓰는 뜻
Release candidate	배포 직전 검증 버전
Smoke test	핵심 경로의 빠른 점검
README	설치·실행·복구 안내
Demo	실제 입력과 결과를 보여주는 시연

release_candidate 입출력

INPUT

README.md

OUTPUT

release_candidate

PROCESS

focused tests → full suite → README

VERIFY

local fixture로 end-to-end 결과와 demo JSON이 생성된다.
build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

Release Candidate와 Vercel Demo 실행 흐름

01

focused tests

02

full suite

03

README

04

demo build

05

human release

마지막 판단은 release_candidate에서 사람이 확인합니다.

release_candidate 정상·실패 계약

정상	——	local fixture로 end-to-end 결과와 demo JSON이 생성된다.
경계	——	build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.
외부 쓰기	——	Release Candidate와 Vercel Demo: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·release_candidate

Release Candidate와 Vercel Demo 파일 지도

입력·fixture

README.md

구현 코드

web-demo/

검증·test

scripts/run_day1_preflight.py

따라하기 notebook

materials/day5/day5_service_lab.ipynb

Release Candidate와 Vercel Demo 개념·코드

Release candidate

배포 직전 검증 버전

```
README.md
```

README

설치·실행·복구 안내

```
scripts/run_day1_preflight.py
```

Smoke test

핵심 경로의 빠른 점검

```
web-demo/
```

Demo

실제 입력과 결과를 보여주는 시연

```
materials/day5/day5_service_lab.ipynb
```

Release Candidate와 Vercel Demo 선택 기준

구분	역할	이번 차시의 판단 기준
Release candidate	focused tests 단계의 입력 기준	결정론 test로 먼저 확인
Smoke test	full suite 단계의 교체 경계	adapter 경계를 확인
README	README 단계의 운영 조건	비용·지연·권한을 확인
Demo	demo build 단계의 판단 기록	사람 판단과 운영 기록을 확인

Release Candidate와 Vercel Demo 대표 실패

실패

Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.

사용자 영향

Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다. 상태에서는 release_candidate을 운영 판단에 사용할 수 없다.

Release Candidate와 Vercel Demo 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

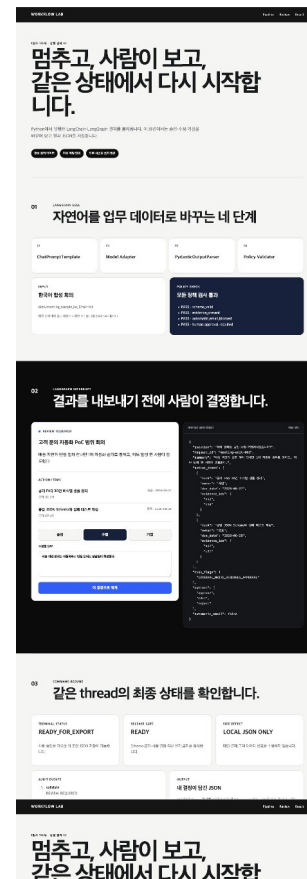
Release Candidate와 Vercel Demo 복구 경로

- 01 — 탐지: build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.
- 02 — 중단: Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.
- 03 — 복구: Python output JSON을 demo의 단일 input으로 사용하고 schema version을 표시한다.
- 04 — 증거: release_candidate과 test 결과를 함께 남긴다.

Release Candidate와 Vercel Demo Codex 검증 루프

web-demo/의 책임을 찾고
release_candidate 계약을
focused test로 고정합니다.

Release Candidate와 Vercel Demo
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Release Candidate와 Vercel Demo Codex 작업 명세

목표

기능 구현 후 README·test·demo data가 같은 contract인지 diff로 검토한다.

허용 경로

web-demo/
scripts/run_day1_preflight.py

완료 조건

local fixture로 end-to-end 결과와 demo JSON이 생성된다.
build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

금지

Release Candidate와 Vercel Demo: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Release Candidate와 Vercel Demo 독립 리뷰

- 01 **Codex diff** — Release Candidate와 Vercel Demo 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — local fixture로 end-to-end 결과와 demo JSON이 생성된다.
- 03 **Claude review** — release_candidate 변경 라인·누락 경계를 별도 session에서 검토
- 04 **Human merge** — Release Candidate와 Vercel Demo의 두 LLM 의견과 test는 자동 승인이 아니다

Release Candidate와 Vercel Demo 정상 시연

- 01 — 열기: README.md
- 02 — 구현 확인: web-demo/
- 03 — 실행: python scripts/run_day1_preflight.py
- 04 — 성공 신호: local fixture로 end-to-end 결과와 demo JSON이 생성된다.

Release Candidate와 Vercel Demo 실행 명령

```
$ python scripts/run_day1_preflight.py
```

실행 전 확인

Release Candidate와 Vercel Demo: repository root · Python 3.12 · .env 미출력

07_release_candidate.json 실행 결과

```
{
  "demo_decisions": {
    "2": "READY",
    "3": "READY_FOR_HUMAN_REVIEW",
    "4": "READY_FOR_SANDBOX_REVIEW",
    "5": "READY"
  },
  "web_build": {
    "command": "/opt/homebrew/bin/node build.mjs",
    "returncode": 0,
    "stdout_tail": [
      "Built dist/ with the review UI and generated workflow data."
    ],
    "stderr_tail": []
  },
  "schema_source": "output/course-demos/dayN/demo_result.json"
}
```

Release Candidate와 Vercel Demo 실패·복구 시연

재현

**build 실패·test 실패·HOLD가 있으면
release candidate가 아니다.**

복구

**Python output JSON을 demo의 단일
input으로 사용하고 schema version을
표시한다.**

확인: build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

Release Candidate와 Vercel Demo 소프트웨어 제작

열 파일	<code>web-demo/ scripts/run_day1_preflight.py</code>
작업 범위	Release Candidate와 Vercel Demo 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	local fixture로 end-to-end 결과와 demo JSON이 생성된다. release_candidate 저장

Release Candidate와 Vercel Demo Notebook 셀

materials/day5/day5_service_lab.ipynb · 7차시

```
import shutil
from src.course_services.course_demo import write_course_demo

release_data = {
    day: write_course_demo(day, workspace_root=ROOT, output_dir=ROOT / f"output/course-demos/day{day}")
    for day in (2, 3, 4, 5)
}
node = shutil.which("node")
web_build = run_command(node, "build.mjs", cwd=ROOT / "web-demo") if node else {
    "command": "node build.mjs", "returncode": 2, "error_code": "NODE_NOT_AVAILABLE"
}
release_candidate = {
    "demo_decisions": {str(day): result["decision"] for day, result in release_data.items()},
    "web_build": web_build,
    "schema_source": "output/course-demos/dayN/demo_result.json",
}
save_json("07_release_candidate.json", release_candidate)
release_candidate
```

Release Candidate와 Vercel Demo Terminal 실행

```
$ python scripts/run_day1_preflight.py
```

저장 위치

```
output/course-labs/day5/07_release_candidate.json
```

local fixture로 end-to-end 결과와 demo JSON이 생성된다.
실패 시 release_candidate의 status·error_code를 먼저 확인

Release Candidate와 Vercel Demo 실패 증거

build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

```
result_file = "output/course-labs/day5/07_release_candidate.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

release_candidate이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Release Candidate와 Vercel Demo 정상 Case

NORMAL

local fixture로 end-to-end 결과와 demo JSON이 생성된다.

- 01 — README.md 입력과 command가 기록됐는가
- 02 — release_candidate schema가 validate되는가
- 03 — Release Candidate와 Vercel Demo focused test가 실제로 통과했는가

Release Candidate와 Vercel Demo 경계 Case

BOUNDARY

build 실패·test 실패·HOLD가 있으면 release candidate가 아니다.

- 01 — Release Candidate와 Vercel Demo: raw traceback 대신 stable error_code가 남는가
- 02 — release_candidate: 외부 쓰기·자동 메일이 false인가
- 03 — build 실패·test 실패·HOLD가 있으면 release candidate가 아니다. 재현이 가능한가

Release Candidate와 Vercel Demo 현업 적용

내부 시연 URL과 재현 가능한 GitHub repo를 한 번에 전달

- 01 — Release Candidate와 Vercel Demo은 합성·비식별 sample로 먼저 실행
- 02 — 내부 시연 URL과 재현 가능한 GitHub repo를 한 번에 전달의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — release_candidate 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — release_candidate을 운영 검토 자료로 사용

Release Candidate와 Vercel Demo 포트폴리오 적용

Release Candidate와 Vercel Demo · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다.의 사용자 영향을 한 문장으로 설명
- 02 — Release Candidate와 Vercel Demo 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 기능 구현 후 README·test·demo data가 같은 contract인지 diff로 검토한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — release_candidate과 README 재현 절차를 제출

Release Candidate와 Vercel Demo 운영 점검

품질

무엇을 local fixture로 end-to-end 결과와 demo JSON이 생성된다.로 정의하는가

복구

Vercel 화면만 동작하고 backend result contract와 다른 mock을 보여준다. 발생 시 release_candidate에서 원인 상태를 확인

안전

Release Candidate와 Vercel Demo의 사람 승인과 external_write=false 위치

관측

release_candidate에서 어떤 status·error_code를 보는가

release_candidate 실행 증거

- 01 — 설명: Release Candidate와 Vercel Demo이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python scripts/run_day1_preflight.py`
- 03 — 증거: release_candidate과 정상·실패 test 결과가 남아 있다.

다음 차시: 최종 Demo와 Scorecard

DAY 5 · 8차시

16:40-17:30 (쉬는 시간 17:30-18:00)

최종 Demo와 Scorecard

Problem · Evidence · final_tag_and_scorecard

진행

최종 Demo와 Scorecard
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

README.md
materials/day5/
day5_service_lab.ipynb

확인할 결과

final_tag_and_scorecard

8차시 입력과 결과

입력	————	release_candidate
이번 차시	————	최종 Demo와 Scorecard
결과	————	final_tag_and_scorecard
다음 연결	————	17:30-18:00 쉬는 시간

최종 Demo와 Scorecard 필요성

기능 목록만 보여주면 왜 안전하고 운영 가능한지 판단할 수 없다.

문제 30초에서 final_tag_and_scorecard까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: final_tag_and_scorecard

최종 Demo와 Scorecard 학습 결과

- 01 — 3분 demo와 2분 질문으로 서비스 가치를 검증한다.
- 02 — 성공 화면만 보여주고 실패·사람 승인·평가를 생략한다. 왜 위험한지 설명한다.
- 03 — final_tag_and_scorecard에서 정상과 실패 상태를 직접 확인한다.

최종 Demo와 Scorecard 용어

용어	이 수업에서 쓰는 뜻
Problem	누구의 어떤 반복 업무인지
Evidence	실행·test·평가 결과
Trade-off	선택으로 얻고 포기한 것
Next step	다음 검증 우선순위

final_tag_and_scorecard 입출력

INPUT

README.md

OUTPUT

final_tag_and_scorecard

PROCESS

문제 30초 → live demo 90초 → 실패·HITL 30초

VERIFY

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

최종 Demo와 Scorecard 실행 흐름

01

문제 30초

02

live demo 90초

03

실패·HITL 30초

04

평가 30초

05

질문 2분

마지막 판단은 final_tag_and_scorecard에서 사람이 확인합니다.

final_tag_and_scorecard 정상·실패 계약

정상	——	처음 보는 사람이 repo에서 demo를 재현할 수 있다.
경계	——	비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
외부 쓰기	——	최종 Demo와 Scorecard: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·final_tag_and_scorecard

최종 Demo와 Scorecard 파일 지도

입력·fixture

README.md

구현 코드

materials/day5/day5_service_lab.ipynb

검증·test

output/day1-workflow/

따라하기 notebook

slides/

최종 Demo와 Scorecard 개념·코드

Problem

누구의 어떤 반복 업무인지

`README.md`

Evidence

실행·test·평가 결과

`materials/day5/day5_service_lab.ipynb`

Trade-off

선택으로 얻고 포기한 것

`output/day1-workflow/`

Next step

다음 검증 우선순위

`slides/`

최종 Demo와 Scorecard 선택 기준

구분	역할	이번 차시의 판단 기준
Problem	문제 30초 단계의 입력 기준	결정론 test로 먼저 확인
Evidence	live demo 90초 단계의 교체 경계	adapter 경계를 확인
Trade-off	실패·HITL 30초 단계의 운영 조건	비용·지연·권한을 확인
Next step	평가 30초 단계의 판단 기록	사람 판단과 운영 기록을 확인

최종 Demo와 Scorecard 대표 실패

실패

성공 화면만 보여주고 실패·사람 승인·평가를 생략한다.

사용자 영향

성공 화면만 보여주고 실패·사람 승인·평가를 생략한다. 상태에서는 `final_tag_and_scorecard`을 운영 판단에 사용할 수 없다.

최종 Demo와 Scorecard 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

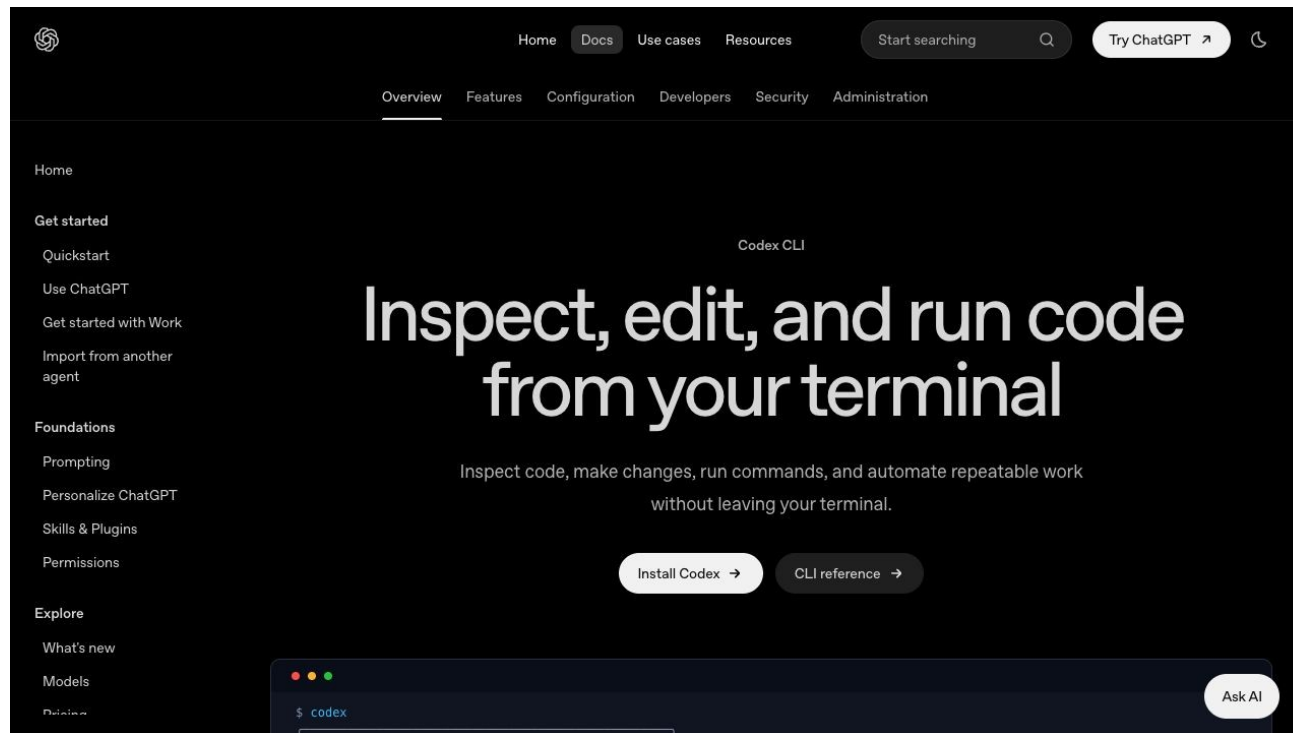
최종 Demo와 Scorecard 복구 경로

- 01 — 탐지: 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.
- 02 — 중단: 성공 화면만 보여주고 실패·사람 승인·평가를 생략한다.
- 03 — 복구: 정상 한 건과 대표 HOLD 한 건을 나란히 보여준다.
- 04 — 증거: final_tag_and_scorecard과 test 결과를 함께 남긴다.

최종 Demo와 Scorecard Codex 검증 루프

materials/day5/
day5_service_lab.ipynb의
책임을 찾고
final_tag_and_scorecard 계약을
focused test로 고정합니다.

최종 Demo와 Scorecard
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



최종 Demo와 Scorecard Codex 작업 명세

목표

발표 전 저장소 구조·README·test 증거를 다시 점검하고 누락만 보완한다.

허용 경로

materials/day5/day5_service_lab.ipynb
output/day1-workflow/

완료 조건

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

금지

최종 Demo와 Scorecard: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

최종 Demo와 Scorecard 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | 최종 Demo와 Scorecard 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 처음 보는 사람이 repo에서 demo를 재현할 수 있다. |
| 03 | Claude review | —— | final_tag_and_scorecard 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 최종 Demo와 Scorecard의 두 LLM 의견과 test는 자동 승인이 아니다 |

최종 Demo와 Scorecard 정상 시연

- 01 — 열기: README.md
- 02 — 구현 확인: materials/day5/day5_service_lab.ipynb
- 03 — 실행: git status --short && python -m pytest -q
- 04 — 성공 신호: 처음 보는 사람이 repo에서 demo를 재현할 수 있다.

최종 Demo와 Scorecard 실행 명령

```
$ git status --short && python -m pytest -q
```

실행 전 확인

최종 Demo와 Scorecard: repository root · Python 3.12 · .env 미출력

08_release_scorecard.json 실행 결과

```
{
  "day": 5,
  "service": "Agent Operations Console",
  "status": "SUCCESS",
  "decision": "READY",
  "input": {
    "kinds": [
      "meeting_transcript",
      "code_diff"
    ],
    "synthetic": true
  },
  "stages": [
    {
      "name": "explicit_router",
      "status": "SUCCESS",
      "artifact": "unified_service_result.json"
    },
    {
      "name": "local_trace",
      "status": "SUCCESS",
      "artifact": "trace.json"
    },
    {
      "name": "dataset_experiment",
      ...
    }
  ]
}
```

최종 Demo와 Scorecard 실패·복구 시연

재현

비밀키·실제 고객 데이터·자동 외부 쓰기는
발표 자료에 없다.

복구

정상 한 건과 대표 HOLD 한 건을 나란히
보여준다.

확인: 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

최종 Demo와 Scorecard 소프트웨어 제작

열 파일 `materials/day5/day5_service_lab.ipynb`
`output/day1-workflow/`

작업 범위 최종 Demo와 Scorecard 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 처음 보는 사람이 repo에서 demo를 재현할 수 있다.
`final_tag_and_scorecard` 저장

최종 Demo와 Scorecard Notebook 셀

materials/day5/day5_service_lab.ipynb · 8차시

```
from src.course_services.course_demo import build_course_demo

final_scorecard = build_course_demo(5, workspace_root=ROOT)
focused_test = run_command(sys.executable, "-m", "pytest", "-q", "tests/test_course_services.py",
"tests/test_observability_workflow.py")
final_scorecard["focused_test"] = focused_test
final_scorecard["web_build"] = web_build
assert final_scorecard["decision"] == "READY"
assert final_scorecard["boundary_case"]["status"] == "HOLD"
assert focused_test["returncode"] == 0
save_json("08_release_scorecard.json", final_scorecard)
{"decision": final_scorecard["decision"], "boundary": final_scorecard["boundary_case"], "metrics":
final_scorecard["metrics"]}
```


최종 Demo와 Scorecard Terminal 실행

```
$ git status --short && python -m pytest -q
```

저장 위치

```
output/course-labs/day5/08_release_scorecard.json
```

처음 보는 사람이 repo에서 demo를 재현할 수 있다.
실패 시 `final_tag_and_scorecard`의 `status.error_code`를 먼저 확인

최종 Demo와 Scorecard 실패 증거

비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

```
result_file = "output/course-labs/day5/08_release_scorecard.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

final_tag_and_scorecard이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

최종 Demo와 Scorecard 정상 Case

NORMAL

처음 보는 사람이 repo에서 demo를 재현할 수 있다.

- 01 — README.md 입력과 command가 기록됐는가
- 02 — final_tag_and_scorecard schema가 validate되는가
- 03 — 최종 Demo와 Scorecard focused test가 실제로 통과했는가

최종 Demo와 Scorecard 경계 Case

BOUNDARY

비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다.

- 01 — 최종 Demo와 Scorecard: raw traceback 대신 stable error_code가 남는가
- 02 — final_tag_and_scorecard: 외부 쓰기·자동 메일이 false인가
- 03 — 비밀키·실제 고객 데이터·자동 외부 쓰기는 발표 자료에 없다. 재현이 가능한가

최종 Demo와 Scorecard 현업 적용

재직자는 업무 적용 계획, 구직자는 제품 판단과 검증 능력을 포트폴리오로 제시

- 01 ——— 최종 Demo와 Scorecard은 합성·비식별 sample로 먼저 실행
- 02 ——— 재직자는 업무 적용 계획, 구직자는 제품 판단과 검증 능력을 포트폴리오로 제시의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 ——— final_tag_and_scorecard 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 ——— final_tag_and_scorecard을 운영 검토 자료로 사용

최종 Demo와 Scorecard 포트폴리오 적용

최종 Demo와 Scorecard · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 성공 화면만 보여주고 실패·사람 승인·평가를 생략한다.의 사용자 영향을 한 문장으로 설명
- 02 ——— 최종 Demo와 Scorecard 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 발표 전 저장소 구조·README·test 증거를 다시 점검하고 누락만 보완한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— final_tag_and_scorecard과 README 재현 절차를 제출

최종 Demo와 Scorecard 운영 점검

품질

무엇을 처음 보는 사람이 repo에서 demo를 재현할 수 있다.로 정의하는가

복구

성공 화면만 보여주고 실패·사람 승인·평가를 생략한다. 발생 시 final_tag_and_scorecard에서 원인 상태를 확인

안전

최종 Demo와 Scorecard의 사람 승인과 external_write=false 위치

관측

final_tag_and_scorecard에서 어떤 status·error_code를 보는가

final_tag_and_scorecard 실행 증거

- 01 — 설명: 최종 Demo와 Scorecard이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git status --short && python -m pytest -q`
- 03 — 증거: final_tag_and_scorecard과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간